Project Proposal: Find the Right Government Resolution

Problem ID: PS000230

Issuing Authority: General Administration Department (GAD)

1. The Official Problem Statement

Finding a specific government resolution is a time-consuming and tedious task. The objective is to develop a Gujarati-English AI assistant that helps users locate and understand relevant Government Resolutions (GRs), circulars, and notifications through questions asked in everyday language. The system must answer using retrieved official documents and provide verifiable evidence, including the supporting document, relevant passage, page number, and document date. It must also identify explicit amendments, replacements, and superseded orders, clearly indicating when available documents cannot establish an answer or its current applicability.

2. In Simple Words: Solving the "Administrative Lag"

In the software world, reading outdated or uncommitted data is known as a "dirty read"—and it causes systems to crash. In government administration, a "dirty read" happens when an officer relies on an outdated, superseded, or partially amended rule.

Currently, government rules exist as a mix of digital PDFs and scanned typewriter-era documents spanning decades. When a citizen or officer asks a question in casual, mixed language (e.g., "What is the maternity leave rule for Class 1 officers?"), standard keyword searches fail. More dangerously, if an officer finds a 2014 rule, they often have no way of knowing if a 2021 notification silently changed it. Our system acts as an absolute truth layer, visually tracing the lineage of a rule from its creation to its current active state, ensuring that no decision is ever made on expired information.

3. The Impact: Why This Matters

Relying on disconnected or outdated circulars causes systemic breakdowns. Here is what happens when administrative lineage is not tracked, and what our tool prevents:

- **The Judicial Overturn Crisis:** If a department penalizes an employee based on a 2015 resolution, only to discover in the Civil Services Tribunal that it was amended in 2018, the state's defense collapses. This leads to administrative embarrassment and wasted legal resources.
- **Financial Recoveries and Audit Penalties:** Establishment branches frequently process allowances based on legacy circulars. During annual Comptroller and Auditor General (CAG) audits, these are flagged as financial irregularities ("Audit Paras"), forcing brutal salary recoveries from lower-level clerks.
- **Citizen Harassment:** When a local desk relies on an obsolete rule, citizens are unjustly denied their welfare benefits or asked for redundant certificates, leading to unnecessary grievance petitions and RTI filings.

Historical context: A famous example of an administrative "dirty read" occurred when the Supreme Court struck down Section 66A of the IT Act, yet local stations continued enforcing it for years because they were reading outdated bare acts without a supersession graph. Our system eliminates this disconnect.

4. Minimum Prototype Requirements

Before implementing any innovative features, our 15-day prototype will flawlessly execute the strict rubric demanded by the GAD over a dataset of 100 to 200 public GRs:

- **Data Extraction:** Automatically extract department, subject, GR number, and issue date from both scanned and digitally generated documents.
- **Bilingual Understanding:** Accurately process questions asked in Gujarati, English, and a casual mix of both.
- **Verifiable Evidence:** Display the exact supporting document, the verbatim passage, and the specific page number alongside every answer.
- **Lineage Tracking:** Explicitly identify references to amendments, replacement orders, and superseded instructions.
- **The Guardrail:** Distinguish established facts from assumptions and proudly state "Insufficient Evidence" when available documents cannot determine current applicability.

5. 8-Person Team Split & Execution Plan

To complete this ambitious build before the November 4 coding deadline, our 8-member team is divided into four specialized, parallel-working squads of two.

Squad Roles:

- **Squad 1 (Data & Ingestion):** Download 150 GRs; handle OCR and metadata extraction.
- **Squad 2 (Core AI & Guardrails):** Build the bilingual search engine and "Insufficient Evidence" guardrail.
- **Squad 3 (Backend & Graph):** Build the API and track the timeline/lineage of amended laws.
- **Squad 4 (Frontend UI):** Build the chat interface, visual amendment timeline, and exact-passage PDF highlighter.

10-Day Sprint Schedule (Oct 26 – Nov 4):

Date	Squad 1 (Data)	Squad 2 (Core AI)	Squad 3 (Backend)	Squad 4 (Frontend)
Oct 26	Setup PDF parsers and download dataset.	Setup vector store and test LLM API keys.	Setup basic server and graph logic basics.	Setup UI boilerplate and layout framework.
Oct 27	Extract metadata (Date, GR No.) from digital PDFs.	Chunk text manually; test basic English search.	Define API schemas (Query to Response).	Build basic chat window (input, bubbles).
Oct 28	Integrate OCR for scanned Gujarati PDFs.	Tune prompt for mixed Gujarati-English queries.	Write logic to detect "supersession" phrases.	Integrate split-screen PDF viewer.
Oct 29	Process full 150-document dataset to JSON.	Focus on forcing the AI to refuse fake rules.	Map GR connections into node/edge graph data.	Build timeline component for dummy data.
Oct 30	**Integration:** Pass dataset to Squad 2.	**Integration:** Ingest data into vector store.	**Integration:** Connect AI engine to API endpoints.	**Integration:** Connect UI to real API data.
Oct 31	Refine metadata and fix OCR errors on messy scans.	Tune pipeline to output exact page numbers.	Structure graph response for the frontend.	Map citation data to auto-scroll the PDF.
Nov 1	Assist Frontend with bounding box coordinates.	Test edge cases to ensure zero hallucinations.	Finalize active vs. superseded flag logic.	Connect coordinates to draw highlight box on PDF.
Nov 2	**Freeze Data.**	**Freeze Prompts.**	**Freeze APIs.**	**Freeze UI.** Polish colors and Trust Shield.
Nov 3	Try to break the search with complex queries.	Monitor latency and hallucination rates.	Check if graph correctly maps a 3-step chain.	Fix visual bugs (text overflow, node overlap).
Nov 4	Draft the document index for the pitch deck.	Write the technical abstract section (AI Engine).	Write the technical abstract section (Architecture).	Do a dry-run of the exact screen clicks for video.

November 5 (Final Day): Absolute code freeze. The entire team will focus solely on recording the screen-capture demo, editing the video, and finalizing the presentation deck for submission.